

EXPERIMENT NO. 4

Sequential Statements • Conditional Branching • Iterative Looping

Question 1 — Sequential Statements

Problem Statement

Write a C program to input two non-zero positive integers and display their sum, difference, product, quotient, remainder, average, and square of their sum.

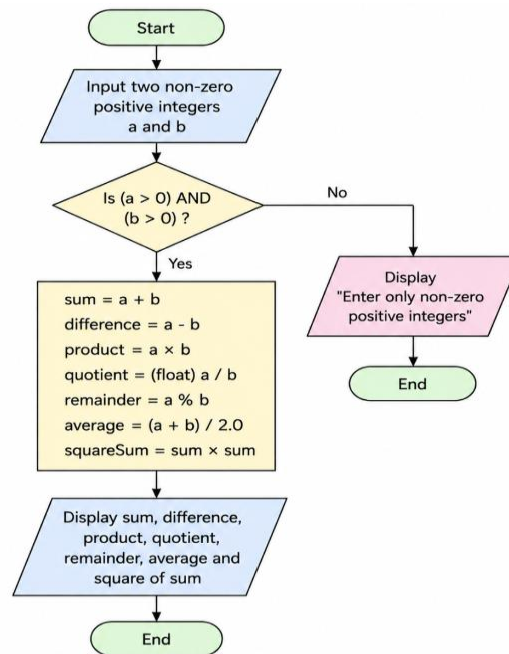
Algorithm

1. Start.
2. Input two non-zero positive integers a and b.
3. Check that both a and b are greater than 0. If not, display an error message and stop.
4. Calculate $\text{sum} = a + b$ and $\text{difference} = a - b$.
5. Calculate $\text{product} = a * b$ and $\text{quotient} = (\text{float})a / b$.
6. Calculate $\text{remainder} = a \% b$ and $\text{average} = (a + b) / 2.0$.
7. Calculate $\text{squareSum} = \text{sum} * \text{sum}$.
8. Display all calculated results.
9. Stop.

Pseudocode

```
START
INPUT a, b
IF a > 0 AND b > 0 THEN
    sum <- a + b
    difference <- a - b
    product <- a * b
    quotient <- a / b
    remainder <- a MOD b
    average <- (a + b) / 2.0
    squareSum <- sum * sum
    DISPLAY sum, difference, product, quotient,
            remainder, average, squareSum
ELSE
    DISPLAY "Enter only non-zero positive integers"
END IF
STOP
```

Flowchart



C Program

```
#include <stdio.h>
```

```
int main(void)
{
    int a, b, sum, difference, product, remainder, squareSum;
    float quotient, average;
```

```
    printf("Enter two non-zero positive integers: ");
    scanf("%d %d", &a, &b);
```

```
    if (a > 0 && b > 0)
    {
        sum = a + b;
        difference = a - b;
        product = a * b;
        quotient = (float)a / b;
        remainder = a % b;
        average = (a + b) / 2.0f;
        squareSum = sum * sum;
```

```
        printf("Sum = %d\n", sum);
        printf("Difference = %d\n", difference);
        printf("Product = %d\n", product);
        printf("Quotient = %.2f\n", quotient);
        printf("Remainder = %d\n", remainder);
        printf("Average = %.2f\n", average);
        printf("Square of sum = %d\n", squareSum);
    }
```

```
    else
    {
        printf("Enter only non-zero positive integers.\n");
    }
```

```
    return 0;
}
```

Compilation and Execution

```
(kali@kali)-[~/lab4]
$ gcc basic.c -o basic
```

```
(kali@kali)-[~/lab4]
$ ./basic
Enter 2 non-zero positive integers:10 5
Sum = 15
Difference = 5
Product = 50
Quotient = 2.00
Remainder = 0
Average = 7.00
Square of Sum = 225
```

Activity 1 – Sequential Statements

Test Case	Objective / Scenario	Input Values	Expected Output	Actual Output	Result
TC-01	Valid input with small numbers	a = 10, b = 5	Sum = 15; Difference = 5; Product = 50; Quotient = 2.00; Remainder = 0; Average = 7.50; Square of sum = 225	Sum = 15; Difference = 5; Product = 50; Quotient = 2.00; Remainder = 0; Average = 7.50; Square of sum = 225	Pass
TC-02	Valid input where remainder is not zero	a = 17, b = 4	Sum = 21; Difference = 13; Product = 68; Quotient = 4.25; Remainder = 1; Average = 10.50; Square of sum = 441	Sum = 21; Difference = 13; Product = 68; Quotient = 4.25; Remainder = 1; Average = 10.50; Square of sum = 441	Pass
TC-03	Both numbers are equal	a = 8, b = 8	Sum = 16; Difference = 0; Product = 64; Quotient = 1.00; Remainder = 0; Average = 8.00; Square of sum = 256	Sum = 16; Difference = 0; Product = 64; Quotient = 1.00; Remainder = 0; Average = 8.00; Square of sum = 256	Pass
TC-04	First number smaller than second	a = 6, b = 12	Sum = 18; Difference = -6; Product = 72; Quotient = 0.50; Remainder = 6; Average = 9.00; Square of sum = 324	Sum = 18; Difference = -6; Product = 72; Quotient = 0.50; Remainder = 6; Average = 9.00; Square of sum = 324	Pass
TC-05	Larger positive numbers	a = 100, b = 25	Sum = 125; Difference = 75; Product = 2500; Quotient = 4.00; Remainder = 0; Average = 62.50; Square of sum = 15625	Sum = 125; Difference = 75; Product = 2500; Quotient = 4.00; Remainder = 0; Average = 62.50; Square of sum = 15625	Pass

Question 2 — Conditional Branching

Problem Statement

Write a menu-driven C program to input a temperature in Celsius and convert it into Kelvin, Fahrenheit, Reaumur, or Rankine based on the user's choice. Use simple IF statements to perform the selected conversion and display the result.

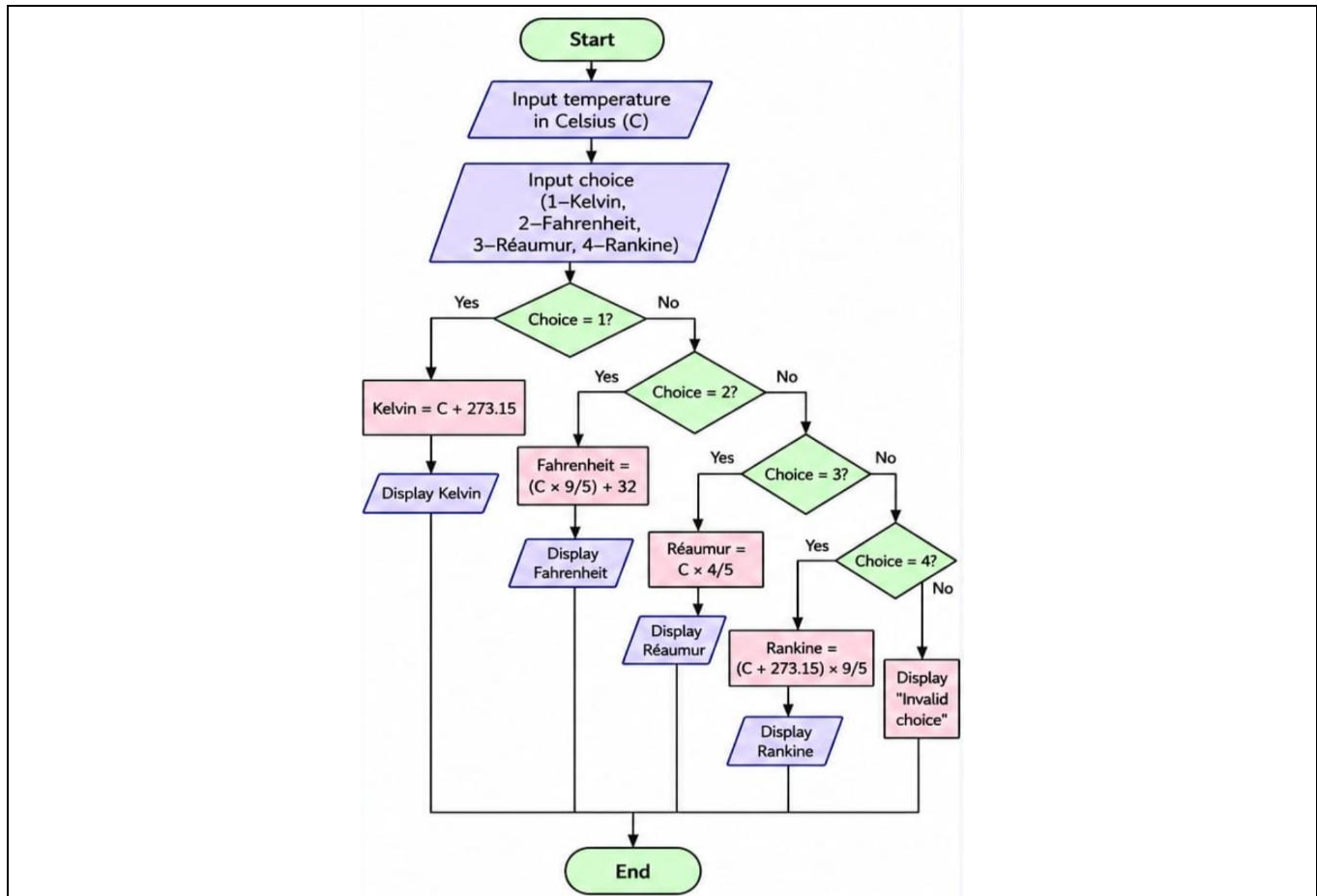
Algorithm

1. Start.
2. Input temperature in Celsius (C).
3. Display the menu: 1-Kelvin, 2-Fahrenheit, 3-Reaumur, 4-Rankine.
4. Input the user's choice.
5. If choice is 1, calculate $K = C + 273.15$ and display it.
6. If choice is 2, calculate $F = (C * 9/5) + 32$ and display it.
7. If choice is 3, calculate $Re = C * 4/5$ and display it.
8. If choice is 4, calculate $R = (C + 273.15) * 9/5$ and display it.
9. If the choice is outside 1 to 4, display "Invalid choice".
10. Stop.

Pseudocode

```
START
INPUT C
DISPLAY "1. Kelvin  2. Fahrenheit  3. Reaumur  4. Rankine"
INPUT choice
IF choice = 1 THEN
    K <- C + 273.15
    DISPLAY K
END IF
IF choice = 2 THEN
    F <- (C * 9 / 5) + 32
    DISPLAY F
END IF
IF choice = 3 THEN
    Re <- C * 4 / 5
    DISPLAY Re
END IF
IF choice = 4 THEN
    R <- (C + 273.15) * 9 / 5
    DISPLAY R
END IF
IF choice < 1 OR choice > 4 THEN
    DISPLAY "Invalid choice"
END IF
STOP
```

Flowchart



C Program

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float celsius, result;
```

```
    int choice;
```

```
    printf("Enter temperature in Celsius: ");
```

```
    scanf("%f", &celsius);
```

```
    printf("1. Kelvin\n");
```

```
    printf("2. Fahrenheit\n");
```

```
    printf("3. Reaumur\n");
```

```
    printf("4. Rankine\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    if (choice == 1)
```

```
    {
```

```
        result = celsius + 273.15f;
```

```
        printf("Temperature in Kelvin = %.2f K\n", result);
```

```
    }
```

```
    if (choice == 2)
```

```
    {
```

```
        result = (celsius * 9.0f / 5.0f) + 32.0f;
```

```
        printf("Temperature in Fahrenheit = %.2f F\n", result);
```

```

}

if (choice == 3)
{
    result = celsius * 4.0f / 5.0f;
    printf("Temperature in Reaumur = %.2f Re\n", result);
}

if (choice == 4)
{
    result = (celsius + 273.15f) * 9.0f / 5.0f;
    printf("Temperature in Rankine = %.2f R\n", result);
}

if (choice < 1 || choice > 4)
{
    printf("Invalid choice.\n");
}

return 0;
}

```

Compilation and Execution

The image displays four terminal windows showing the compilation and execution of the program. The first window shows the source file 'temp.c' being compiled with 'gcc temp.c -o temp'. The subsequent three windows show the program being run with different choices (1, 2, and 4) and the corresponding temperature conversions.

```

(kali㉿kali)-[~/lab4]
$ vi temp.c

(kali㉿kali)-[~/lab4]
$ gcc temp.c -o temp

(kali㉿kali)-[~/lab4]
$ ./temp
Enter temperature in Celsius: 25
1. Kelvin
2. Fahrenheit
3. Reaumur
4. Rankine
Enter your choice: 1
Temperature in Kelvin = 298.15 K

(kali㉿kali)-[~/lab4]
$ ./temp
Enter temperature in Celsius: 25
1. Kelvin
2. Fahrenheit
3. Reaumur
4. Rankine
Enter your choice: 2
Temperature in Fahrenheit = 77.00 F

(kali㉿kali)-[~/lab4]
$ ./temp
Enter temperature in Celsius: 25
1. Kelvin
2. Fahrenheit
3. Reaumur
4. Rankine
Enter your choice: 3
Temperature in Reaumur = 20.00 Re

(kali㉿kali)-[~/lab4]
$ ./temp
Enter temperature in Celsius: 25
1. Kelvin
2. Fahrenheit
3. Reaumur
4. Rankine
Enter your choice: 4
Temperature in Rankine = 536.67 R

```

Activity 2 – Conditional Branching

Test Case	Objective / Scenario	Input Values	Expected Output	Actual Output	Result
TC-01	Convert Celsius to Kelvin	C = 25, Choice = 1	Temperature in Kelvin = 298.15 K	Temperature in Kelvin = 298.15 K	Pass
TC-02	Convert Celsius to Fahrenheit	C = 25, Choice = 2	Temperature in Fahrenheit = 77.00 F	Temperature in Fahrenheit = 77.00 F	Pass
TC-03	Convert Celsius to Reaumur	C = 25, Choice = 3	Temperature in Reaumur = 20.00 Re	Temperature in Reaumur = 20.00 Re	Pass
TC-04	Convert Celsius to Rankine	C = 25, Choice = 4	Temperature in Rankine = 536.67 R	Temperature in Rankine = 536.67 R	Pass
TC-05	Invalid menu choice	C = 25, Choice = 5	Invalid choice.	Invalid choice.	Pass

Question 3 — Iterative Looping

Problem Statement

Write a C program to input a positive integer and find the number of digits using the formula “digits = $\log_{10}(\text{num}) + 1$ ”. Using a FOR loop, extract each digit of the number and display the sum and product of its digits.

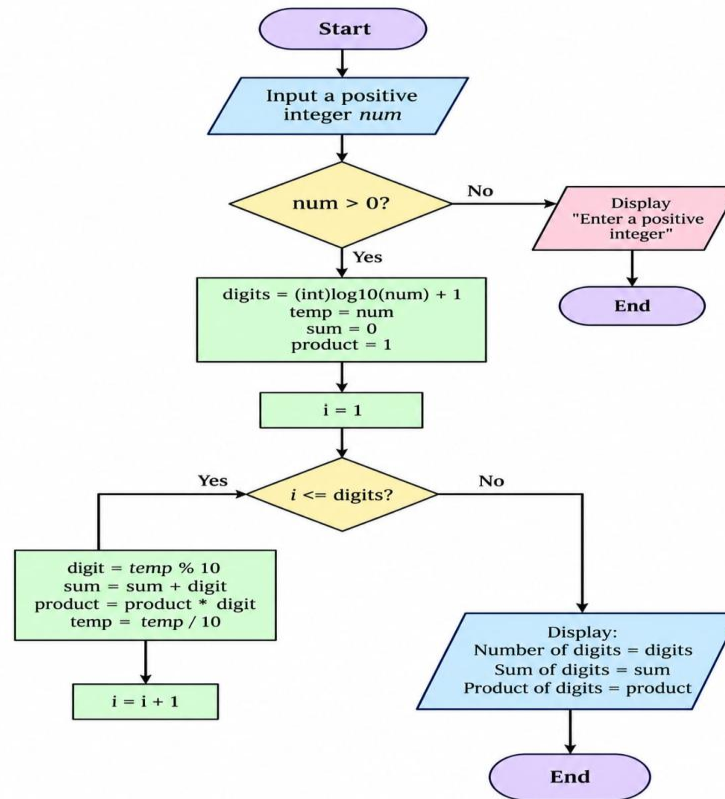
Algorithm

1. Start.
2. Input a positive integer num.
3. If num is not positive, display an error message and stop.
4. Calculate $\text{digits} = (\text{int})\log_{10}(\text{num}) + 1$.
5. Set $\text{temp} = \text{num}$, $\text{sum} = 0$, and $\text{product} = 1$.
6. Repeat for $i = 1$ to digits.
7. Extract $\text{digit} = \text{temp} \% 10$.
8. Update $\text{sum} = \text{sum} + \text{digit}$ and $\text{product} = \text{product} * \text{digit}$.
9. Remove the last digit using $\text{temp} = \text{temp} / 10$.
10. After the loop, display digits, sum, and product.
11. Stop.

Pseudocode

```
START
INPUT num
IF num > 0 THEN
    digits <- (INTEGER) log10(num) + 1
    temp <- num
    sum <- 0
    product <- 1
    FOR i <- 1 TO digits DO
        digit <- temp MOD 10
        sum <- sum + digit
        product <- product * digit
        temp <- temp / 10
    END FOR
    DISPLAY digits, sum, product
ELSE
    DISPLAY "Enter a positive integer"
END IF
STOP
```

Flowchart



C Program

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    int num, temp, digits, digit, sum = 0, product = 1, i;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num > 0)
    {
        digits = (int)log10(num) + 1;
        temp = num;

        for (i = 1; i <= digits; i++)
        {
            digit = temp % 10;
            sum = sum + digit;
            product = product * digit;
            temp = temp / 10;
        }

        printf("Number of digits = %d\n", digits);
        printf("Sum of digits = %d\n", sum);
        printf("Product of digits = %d\n", product);
    }
    else
    {
        printf("Enter a positive integer.\n");
    }

    return 0; }
```


Compilation and Execution

```
(kali㉿kali)-[~/lab4]
$ gcc digit.c -o digit -lm

(kali㉿kali)-[~/lab4]
$ ./digit
Enter a positive integer: 7
Number of digits = 1
Sum of digits = 7
Product of digits = 7

(kali㉿kali)-[~/lab4]
$ ./digit
Enter a positive integer: 105
Number of digits = 3
Sum of digits = 6
Product of digits = 0
```

```
(kali㉿kali)-[~/lab4]
$ ./digit
Enter a positive integer: 1234
Number of digits = 4
Sum of digits = 10
Product of digits = 24

(kali㉿kali)-[~/lab4]
$ ./digit
Enter a positive integer: 234
Number of digits = 3
Sum of digits = 9
Product of digits = 24

(kali㉿kali)-[~/lab4]
$ ./digit
Enter a positive integer: 98765
Number of digits = 5
Sum of digits = 35
Product of digits = 15120
```

Test Case	Objective / Scenario	Input Values	Expected Output	Actual Output	Result
TC-01	Single-digit number	num = 7	Number of digits = 1; Sum of digits = 7; Product of digits = 7	Number of digits = 1; Sum of digits = 7; Product of digits = 7	Pass
TC-02	Three-digit number	num = 234	Number of digits = 3; Sum of digits = 9; Product of digits = 24	Number of digits = 3; Sum of digits = 9; Product of digits = 24	Pass
TC-03	Number containing zero	num = 105	Number of digits = 3; Sum of digits = 6; Product of digits = 0	Number of digits = 3; Sum of digits = 6; Product of digits = 0	Pass
TC-04	Four-digit number	num = 1234	Number of digits = 4; Sum of digits = 10; Product of digits = 24	Number of digits = 4; Sum of digits = 10; Product of digits = 24	Pass
TC-05	Larger positive integer	num = 98765	Number of digits = 5; Sum of digits = 35; Product of digits = 15120	Number of digits = 5; Sum of digits = 35; Product of digits = 15120	Pass

Result

Thus, C programs based on sequential statements, conditional branching using simple IF statements, and iterative looping using a FOR loop were designed using algorithms, pseudocode, flowcharts, and successfully implemented.